Hack to Scale Single-Leader Replication: Query Logic in Application Layer

1. What Is This Idea?

When one SQL database becomes too large or overloaded, instead of using a native distributed database, we manually split data across multiple SQL databases.

Then the **application** decides:

```
None
Which DB to query?

Which table contains data?

How to merge results?
```

This moves database intelligence into application code.

2. Core Concept

Instead of:

```
None
App → One SQL DB
```

Use:

```
App → DB1

App → DB2

App → DB3
```

Application routes requests based on custom logic.

3. Why People Do This

Because teams want to keep using familiar SQL systems like:

- MySQL
- PostgreSQL

Without migrating immediately to distributed databases.

4. Example

Search table grows billions of rows daily.

Instead of one giant table:

```
fact_searches
```

Split by day:

```
None
fact_searches_2026_04_28

fact_searches_2026_04_29

fact_searches_2026_04_30
```

Now app chooses table by date.

5. Another Example: Customer-Based Split

Heavy enterprise customers generate huge traffic.

Create separate partitions:

```
None
searches_companyA

searches_companyB

searches_companyC
```

Application routes by tenant/customer.

6. What the App Must Handle

A. Routing Logic

```
None
If date = today → DB3

If user = EU → DB2

If customer = A → DB5
```

B. Metadata Management

Need mapping tables:

```
None
Which shard has what data?
```

C. Cross-Database Queries

If query spans many DBs:

```
None
Fetch from DB1 + DB2 + DB3

Merge in application

Sort/Paginate manually
```

7. Why This Is Called a Hack

Because databases already solve query planning internally.

Here:

```
None
SQL logic spills into app layer
```

So architecture becomes harder to maintain.

8. Major Problems

A. Complexity Explosion

More DBs + more routing code.

B. Hard Joins

Joining across DBs becomes painful.

C. Transactions Harder

Cross-database ACID is difficult.

D. Duplicate Logic

Many services may need same routing rules.

E. Rebalancing Pain

Moving data between DBs is manual.

9. Example of Query Difficulty

Need monthly report:

```
None
SELECT count(*) FROM all April data
```

Now app must query 30 tables or many databases and combine totals.

10. Better Alternatives

Prefer systems designed for scale:

- Apache Cassandra
- CockroachDB
- Google Spanner
- Vitess
- Citus

11. When This Approach Is Acceptable

Use temporarily when:

```
Need quick scale now

Existing SQL expertise strong

Migration budget low

Simple partition key exists
```

12. HLD Interview Framing

Say:

> As an interim solution, we can shard across multiple SQL databases with routing logic in the application layer, but long term I'd prefer distributed SQL or native sharding middleware.

Strong answer because it shows pragmatism.

13. Real Industry Examples

Many companies started this way before adopting:

- Vitess
- PlanetScale

14. Short Revision Notes

```
None
Manual SQL Scaling:


Split data across DBs

App decides routing


Pros:

fast workaround

keep SQL


Cons:

complexity

joins hard

transactions hard

maintenance costly
```

15. One-Line Summary

Manually partitioning SQL databases and moving query routing into the application can extend single-node SQL systems temporarily, but significantly increases long-term complexity and maintenance cost.